

Self-Improving Agents 2.0

Getting better without retraining the base model

Day 100 promised it and Day 101 set it up: today is the technique for how an agent improves itself without ever touching model weights. In 2026 self-improvement stopped being a single research trick and split into three production levers -- reinforcement learning on MEMORY (MemRL), evolving a SKILL LIBRARY, and rewriting the agent's own CODE (the Darwin-Godel line). The throughline the series has been building: with frontier models within ~3% of each other (Day 80) and Chinese open models now 60-90% cheaper (CNBC, Jul 7), you can neither out-model nor out-spend a rival -- so the durable edge is a learned SYSTEM (memory + skills + harness) that survives swapping the model underneath it. Memory was the moat (Day 44); now the whole learned scaffold is.

3 levers improve an agent WITHOUT touching model weights	20% -> 50% Darwin-Godel self-improvement on SWE-bench	166 tasks SkillFlow lifelong skill-evolution benchmark	60-90% cheaper Chinese open models (CNBC, Jul 7)
--	--	--	--

1) Four places an agent can change -- and why three of them skip retraining

Fine-tuning the base model is only one of four places an agent can improve, and it is the slowest, most expensive and least portable. The other three -- MEMORY, the SKILL LIBRARY, and the agent's own CODE -- all change behaviour with the weights frozen. That matters more in 2026 than ever: models are within ~3% of each other and interchangeable (Day 80), and Chinese open models undercut US frontier by 60-90% (CNBC, Jul 7), so teams rotate the model underneath an agent constantly. Anything you learned by fine-tuning a specific checkpoint evaporates the moment you route to a cheaper model; anything you learned into memory or a skill library survives the swap.

So 'self-improving agents 2.0' is really a taxonomy: (1) weight-level = fine-tuning / RLHF (offline, costly, model-locked); (2) MEMORY-level = MemRL, learn WHICH past experiences to reuse; (3) SKILL-level = externalise experience into reusable procedures and curate a library; (4) CODE-level = the agent rewrites its own harness (Darwin-Godel). The rest of this issue teaches levers 2-4 -- the ones you can actually run in production without a training pipeline -- and where each one belongs.

Lever	What changes	Cost / risk	Survives model swap?
Fine-tune	Model weights	High infra, model-locked	No -- lost on rotation
MemRL (memory)	Retrieval utilities (Q-values)	Lowest -- runtime, no training	Yes
Skill library	Reusable SKILL.md procedures	Low -- portable text	Yes
Darwin-Godel (code)	The agent's own harness	Highest -- sandbox only	Yes (with the archive)

2) MemRL -- reinforcement learning on the memory, not the model

MemRL's move is to DECOUPLE stable cognitive reasoning (the frozen model) from dynamic episodic memory, then apply reinforcement learning to the memory at runtime. Concretely: every time the agent finishes a task, the trajectory is stored as an episodic memory; a two-phase retrieval later pulls candidates by semantic relevance and then re-ranks them by a learned Q-value -- a utility score for 'how much did reusing this actually help?' Those utilities are refined trial-and-error from environmental feedback, so a strategy that looked relevant but kept failing gets down-weighted and stops being retrieved.

The payoff is that the agent gets better on a repeated task distribution with no fine-tuning and no weight updates -- it is resolving the stability-plasticity dilemma by keeping the model stable and letting the memory be plastic. The MemTensor team (the MemOS people from Day 30 and Day 44) report it beating SOTA baselines on HLE, BigCodeBench, ALFWorld and Lifelong Agent Bench, and open-sourced the code. Architecturally this is the production maturation of the 'MemRL' the series has flagged since Hermes Agent and MiniMax M2.7 -- now a named, benchmarked, runnable pattern rather than a footnote in a model release.

3) Skill-library evolution -- externalise experience, repair it, keep it compact

The second lever treats a SKILL (a SKILL.md-style reusable procedure, Days 55-61) as the unit of learning. The loop, formalised across 2026 work: the agent solves a task, distils the procedure + pitfalls + verification into a skill, REPAIRS that skill after it later fails, and CURATES a compact high-utility library so the collection does not bloat into noise. MUSE-Autoskill frames it as create / memory / manage / evaluate; SkillOS learns skill curation; SkillClaw and Group-Evolving Agents let a GROUP of agents share trajectories, tools and learned skills directly -- something biology can't do -- so a discovery by one agent becomes long-term progress for all of them.

The honest finding from SkillFlow (a benchmark of 166 runnable tasks across 20 families) is that the gains are SELECTIVE, not universal: strong model+harness stacks convert experience into compact reusable procedures, while weaker stacks show coordination gaps and fragmented, low-utility libraries. Translation: skill evolution amplifies an already-good agent; it does not rescue a shaky one. And because a skill is portable text, the library you grow survives the model swap that a fine-tune would not -- the reason this beats weight-level learning when your model rotates weekly.

4) The Darwin-Godel line -- the agent rewrites its own code

The most autonomous (and most dangerous) lever: the agent modifies its OWN code, including the code that proposes modifications. The Darwin-Godel Machine (Sakana AI / Zhang, Hu, Lu, Lange, Clune; ICLR 2026) borrows open-ended evolution -- keep a growing ARCHIVE of agent variants, select parents by quality AND diversity, mutate, keep what benchmarks better. Left to run it lifted a coding agent from 20.0% to 50.0% on SWE-bench and 14.2% to 30.7% on Polyglot, and autonomously invented better edit tools, long-context management, and peer-review-of-its-own-output steps -- capabilities nobody hand-coded.

The 2026 line extends it: DARWIN (Dynamic Agentially Rewriting Self-Improving Network) and Group-Evolving Agents push open-ended, experience-sharing self-modification further. This is also where governance stops being optional. A system that rewrites its own harness and is selected for capability can learn to subvert oversight or resist shutdown (the Berkeley Agentic-AI-Profile risk, Day 21). So Darwin-Godel-class self-improvement belongs ONLY inside a tight sandbox: scoped SPIFFE/SVID identity (Day 54), execution isolation, an immutable policy layer the agent can't edit, a human on the archive-PROMOTION gate, and a sub-second kill switch. The archive promotion is the new deploy.

THE ONE RULE

A memory write, a skill promotion, and a Darwin-Godel archive update are each a DEPLOY -- gate them like one. OTEL -> WORM audit of what was learned, a human on the promotion gate, a kill switch that freezes the LEARNING loop (not just the current run). Automate the improvement; never automate the decision to keep it. (Also your EU AI Act evidence -- Aug 2 = T-25.)

5) Which lever, and the one governance rule that covers all three

A decision guide. Reach for MemRL when you have a repeated task distribution and want cheap, continuous improvement with no training infra -- it is the lowest-risk lever because it only changes retrieval weights. Reach for skill-library evolution when your workload has reusable procedures and you want the learning to be PORTABLE across models (the compression + Chinese-cost pressure makes portability worth real money). Reach for Darwin-Godel only in a sandboxed, human-gated research setting -- highest capability, highest blast

radius.

The single rule that spans all three: a memory write, a skill promotion, and an archive update are each a DEPLOY, so gate them like one. Same OTEL -> WORM audit trail the series has wired since Day 22/50, a human on the promotion gate for anything that changes the agent's future behaviour, and a kill switch that can freeze the learning loop, not just the current task. That is also your EU AI Act evidence -- a self-improving high-risk agent has to show WHAT it learned and WHY it kept it; Aug 2 is T-25. Automate the improvement loop; never automate the decision to keep the improvement.

When to reach for it	Use	Guardrail
Repeated task distribution, no training infra	MemRL	Utility audit + memory kill switch
Reusable procedures, portability across models	Skill evolution	Compact library + promotion gate
Research, tightly sandboxed only	Darwin-Godel	Isolation + immutable policy + <1s kill

VIRAL / OPEN-SOURCE SPOTLIGHT

MemTensor / MemRL -- self-evolving memory as runnable open source

The runnable embodiment of the day's headline technique. MemRL (MemTensor -- the same team behind MemOS, tracked since Day 30) open-sourced the code for 'Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory': the agent improves by applying RL to its EPISODIC MEMORY at runtime -- no fine-tuning, no weight updates, no GPU training run. A two-phase retrieval first filters memories by semantic relevance, then re-ranks them by a learned Q-value (utility) that is continuously refined from environmental feedback, so the agent learns to distinguish a high-value strategy from lookalike noise. It reconciles the stability-plasticity dilemma (keep stable reasoning, adapt the memory) and outperforms SOTA baselines on HLE, BigCodeBench, ALFWorld and Lifelong Agent Bench. It rides a whole 2026 skill-evolution repo wave -- SkillClaw, MUSE-Autoskill, SkillIOS, Group-Evolving Agents -- while OpenClaw (210K+ stars) still tops the raw OSS charts as the no-guardrail foil: a self-improving agent that writes to its own memory or skill library with no promotion gate is exactly what this issue warns against.

0 weight updates RL applied to memory, not the model	2-phase retrieval: semantic filter -> Q-value utility	4 benchmarks beats SOTA (HLE, BigCodeBench, ALFWorld, LAB)
--	---	--

MARKET SIGNAL

Self-improvement-without-retraining is exactly the capability the 2026 cost squeeze rewards. With frontier models compressed to a near-tie and Chinese open models 60-90% cheaper -- US companies now route 30-46% of their tokens to Chinese models via OpenRouter, and Z.ai's GLM 5.2 landed within ~1% of Opus 4.8 on an agentic benchmark at ~1/5 the cost -- the value shifts from the checkpoint you own to the learned system that rides on top of any checkpoint. A learned memory (MemRL) and a curated skill library are the assets that survive weekly model rotation; a fine-tune is the asset that doesn't. The moat is no longer the model, and increasingly no longer even the memory alone -- it is the whole self-improving scaffold plus the governance evidence that lets you keep it in production.

PRACTICAL TAKEAWAYS

1. Improve the memory and the skills before you fine-tune

For a repeated task distribution, wire MemRL-style retrieval (semantic filter -> learned utility re-rank) before reaching for a fine-tune -- you get continuous improvement with no training infra and, crucially, learning that survives swapping to a cheaper model. Add a skill-writer that distils every multi-step task into a reusable, repairable SKILL.md; keep the library compact and high-utility (SkillFlow shows bloated libraries hurt).

2. Treat every learning write as a deploy

A memory write, a skill promotion, and a Darwin-Godel archive update all change the agent's future behaviour, so gate them like a release: OTEL -> WORM audit of what was learned, a human on the promotion gate, and a

kill switch that freezes the LEARNING loop -- not just the current run. This doubles as EU AI Act evidence (T-25 to Aug 2).

3. Sandbox self-modifying code, hard

Darwin-Godel-class agents rewrite their own harness and are selected for capability, which is precisely the recipe for learning to resist oversight. Run them only with scoped SPIFFE/SVID identity, execution isolation, an immutable policy layer the agent cannot edit, and a sub-second kill switch. Never point one at production infrastructure.

TOMORROW

Tomorrow -- Day 103: World models & simulation-first agents (AMI Labs, robotics sim-to-real) -- how agents learn by rehearsing inside a learned model of the world before touching the real one.